Recommendation Systems for Personalized Content Discovery

Netflix Prize Dataset | Funk SVD vs Neural Collaborative Filtering

Cult Open Projects 2026 — AI/ML Track

Problem Understanding

- Recommendation systems power personalization on Netflix, Spotify, Amazon, and YouTube
- Core challenge: learn what a user likes from few past interactions and predict what they will enjoy next
- Dataset: over 100 million ratings from ~480,000 users across 17,770 movies
- Ratings are on a 1-5 star scale with timestamps
- Matrix is extremely sparse — only 1.18% of all possible user-movie pairs have a rating
- Most users rated only a small fraction of the catalog
- Goal: generate genuinely useful recommendations, not just accurate star predictions
- Two approaches compared: Funk SVD (classical) and Neural Collaborative Filtering (deep learning)
- Key challenge: the matrix is 98.82% empty. Neighbourhood-based collaborative filtering fails because most user pairs share very few co-rated movies. Latent factor models are needed instead.

Exploratory Data Analysis

- Data sparsity: 480,189 users × 17,770 movies = 8.5B possible ratings; only 100.5M observed (1.18% density)
- Average user rated ~209 movies, but varies enormously — many rated fewer than 50
- Rating distribution is positively skewed: 4-star most common (36%), 5-star next (30%)
- 1-star and 2-star ratings are rare (~4% and ~9%) — users rate movies they chose to watch
- Global mean rating is ~3.6, used as a bias anchor in the NCF model
- User activity is heavily right-skewed: top 1% contribute disproportionately
- ~31% of users have fewer than 50 ratings — cold-start problem
- Movie popularity is long-tail: top 500 movies (~2.8%) get ~40% of all ratings
- Density filtering applied: only users and movies with 10-15+ ratings kept for training
- Without bias correction, models systematically over-recommend popular blockbusters and ignore niche titles. Density filtering removes noisy single-rating outliers and concentrates learning on better-evidenced interactions.

Methodology

- Raw Netflix Prize files parsed line by line: movie ID lines end with colon, followed by Customer ID, Rating, Date
- Memory-optimized types: Movie IDs as int16, Customer IDs as int32, Ratings as int8
- Dates parsed as datetime objects; IDs encoded to contiguous integer indices for embedding layers
- 80/20 train-test split with random_state=42 for reproducibility; same test set for both models
- Funk SVD: biased matrix factorization via scikit-surprise; 12-15 latent factors, 30-35 epochs SGD
- SVD learning rate 0.005-0.007, regularization 0.12-0.15 to prevent overfitting on sparse data
- NCF: hybrid architecture with MF path (dot product) + MLP path (feature crossing)
- NCF uses 16-dim embeddings with L2 reg, two dense layers (32 and 16 units), ReLU, dropout 0.1
- NCF trained with MSE loss, Adam optimizer lr=0.001, batch size 16, 30 epochs
- ReduceLROnPlateau halves lr when loss plateaus for 2 epochs, minimum lr 1e-5
- The feature crossing step (element-wise product of embeddings before MLP) lets NCF learn which dimensions of user taste align with movie characteristics — something pure dot products cannot capture.

Model Design

- Funk SVD: predicted rating = global mean + user bias + movie bias + (user vector · movie vector)
- User bias captures generous vs harsh raters; movie bias captures universally loved vs disliked films
- Low latent factors (12-15) deliberate choice: higher dimensions overfit on sparse data
- High regularization penalizes large parameters; goal is broad reliable patterns, not fine-grained personalization
- SVD recommendation: predict for all unseen movies, sort descending, return top K
- NCF: dual-path design — MF path + MLP path summed together with bias terms and global mean
- Global mean anchor (Lambda layer) initializes predictions near data mean, stabilizing early training
- Output clipped to [1.0, 5.0] to prevent impossible predictions and corrupted top-K sorting
- Neither path alone performs as well as the combination — MF gives linear signal, MLP gives non-linear capacity
- The biased formulation is essential. Without per-user and per-movie offsets, the model treats all deviations from global mean as preference signal, introducing systematic error for strict critics and blockbuster films.

Evaluation Metrics

- Both models evaluated on same held-out 20% test partition (random_state=42)
- RMSE: measures rating prediction accuracy — square root of average squared prediction error
- RMSE penalizes large errors quadratically; occasional very wrong predictions hurt more than consistent moderate errors
- Netflix Prize Cinematch baseline RMSE ≈ 1.06; below 1.0 on dense filtered subset is strong
- MAP@10: measures recommendation ranking quality — whether top 10 items are actually relevant
- Relevance threshold: actual rating ≥ 3.5 stars
- For each user: sort test items by predicted rating, take top 10, compute Average Precision
- MAP@10 = mean of per-user AP values; users with no relevant test items excluded
- MAP@10 is more important in practice: users care about top-of-list quality, not background rating precision
- A model can predict 3.7 vs 3.9 stars with high precision and still produce a poorly ordered top-10 list if it ranks mediocre items above genuinely relevant ones.

Results & Recommendation Quality

- Funk SVD: RMSE ~0.92-0.95, MAP@10 ~0.08-0.13, trains in 2-6 seconds
- Neural CF: RMSE ~0.88-0.93, MAP@10 ~0.10-0.16, trains in 30-90 seconds
- NCF consistently higher MAP@10: non-linear feature crossing captures preference patterns SVD misses
- SVD is fast and interpretable — latent factors and biases have clear meaning
- NCF is a black box but produces better-ranked lists — the metric that matters most in deployment
- Density filtering significantly improves both models vs training on raw full data
- SVD recommendations are conservative and well-calibrated, clustering near user's historical average
- NCF recommendations are more varied, picking up on combination effects between embedding dimensions
- For users with 50+ ratings, both produce meaningful personalized picks; for <10 ratings, both fall back to global popularity
- Cold-start remains the main unresolved limitation for both approaches
- NCF identifies its own success case (best prediction) and failure case (largest error) per user. This reveals where the model goes wrong — e.g., overestimating for contrarian users who dislike popular films.

Key Insights & Future Work

- RMSE does not equal ranking quality — optimizing star prediction accuracy does not guarantee good top-10 lists
- Bias modeling is non-negotiable — per-user and per-movie offsets are essential for handling rating biases
- Feature crossing improves NCF over naive concatenation-only architecture
- Density filtering is as important as model choice — quality data beats quantity data
- Output bounding prevents inference artefacts; dynamic LR scheduling helps NCF converge better
- SVD++ with implicit feedback: use which movies were viewed, not just rated — typical 10-20% MAP@10 improvement
- Graph Neural Networks (LightGCN): propagate embeddings through interaction graph for higher-order signals
- Temporal-aware embeddings: weight recent ratings more to capture shifting tastes over time
- Cold-start via content features: use genre, year, cast to bootstrap new movies and users
- Approximate nearest neighbor retrieval (FAISS) for real-time production-scale inference
- Diversity-aware re-ranking (Maximal Marginal Relevance) to reduce redundancy and surface niche items
- Explainable recommendations: human-readable reasons improve trust and regulatory compliance
- For a real product: deploy NCF for ranking quality, add FAISS for millisecond retrieval, and invest heavily in cold-start solutions — new users represent the highest churn risk.